A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

Abstract

Explanation requires ground truth: to verify that an account of a system is correct, we must know the system's inner functioning. This is precisely what is missing wherever explainable AI (XAI) is most needed. The systems we can study today fall into two camps. Either they are simple and procedural— decision trees, rule lists, sparse linear models—where the mechanism is known but trivial, so explaining it tests nothing, or they are genuinely complex—deep networks, real-world tasks—where XAI is needed but no ground truth of the inner functioning exists, so an explanation can be plausible, confident, and wrong with no way to tell. We set out to remove this dichotomy by building the foundation for a study object that is at once genuinely complex and fully known—and, so that gradient-based methods can apply, fully differentiable. We re-implement the Atari 2600 Video Computer System (VCS)—a real computer architecture, and the platform on which deep reinforcement learning was first established—as two independent, end-to-end *differentiable* emulators, one in Julia (JUTARI) and one in JAX (JAXTARI), each validated bit-for-bit against the XITARI reference. Both ports reproduce XITARI exactly on all 64 supported Arcade Learning Environment (ALE) games: 64/64 byte-identical RAM and 64/64 pixel-identical screens. Treating the cartridge ROM as a weight tensor, the RAM as a soft tape, and control flow as gates, we prove that the differentiable (soft) execution is bit-exact equal to the original (hard) execution in the forward pass at any finite temperature, while exposing gradients where bit logic has none. The system was built in roughly 137 hours of active work over 29 calendar days, with large parts written autonomously by coding agents. This paper builds and validates the foundation, and shows— theoretically and in a qualitative gradient study—that gradient-based XAI on it is feasible.

1 Introduction

When we explain a system—in explainable AI (XAI) as anywhere else—we claim that *this* part is responsible for *that* behaviour. Checking such a claim requires knowing the inner functioning independently, as ground truth. Without it we can ask whether an explanation is plausible or convincing, but not whether it is *true*. Ground truth is the prerequisite for verification, and it is exactly what is missing where explanation matters most.

The systems available to study fall into two unsatisfying camps. Simple, procedural models—decision trees, rule lists, sparse linear models—have fully known inner functioning, but there the explanation is essentially the model itself, testing an XAI method on nothing it could get wrong. These are not the systems XAI was created for. The systems XAI does target—deep neural networks, learned agents, real-world pipelines—are genuinely complex and, for that very reason, have no ground truth of their inner functioning, so an attribution or saliency map can be plausible, confident, and wrong with no way to tell (Atrey, Clary, and Jensen 2020; Nikulin et al. 2019).

Neuroscience faces a similar problem. Therefore, Jonas and Kording (2017) asked whether the standard systems-neuroscience toolkit—connectomics, tuning curves, lesioning, dimensionality reduction—could recover how a MOS 6502 microprocessor computes, using the chip as a model organism precisely because its mechanism is complex *and* completely known. Even with perfect, unlimited data the methods produced structure that looked meaningful but did not recover the processor's actual organisation. The conclusion was that the *methods* were lacking, and that the way to find out is to test them on a system whose ground truth we already possess. Two things make it difficult to translate this to modern XAI: the microprocessor was not *differentiable*, so today's gradient-based methods could not be applied, and the analyses were classical statistics, not XAI. The MOS 6502 is no accident—it is the CPU at the heart of the Atari 2600 Video Computer System (VCS), the platform on which modern deep reinforcement learning was first demonstrated (Mnih et al. 2013, 2015; Bellemare et al. 2013).

We therefore build the missing piece: a study object at once *complex* (a real computer architecture), *fully known* to us bit-for-bit, and *differentiable*, so the gradient-based XAI toolkit can in principle be turned on it. We re-implement the VCS—a 6507 CPU executing code from a cartridge ROM, a Television Interface Adapter (TIA) that turns register writes into pixels cycle by cycle, and a RIOT (RAM, I/O, timer) chip providing the RAM, an interval timer, and joystick and switch I/O—as two independent, end-to-end differentiable emulators: JUTARI in Julia (Bezanson et al. 2017) and JAXTARI in JAX (Bradbury et al. 2018). Each runs in a bit-exact *hard*

mode and a differentiable *soft* mode, validated against XITARI (DeepMind 2016), the Stella-derived C++ emulator (Stella Team 2024) that served as the reference for the original deep-Q-network (DQN) work.

This paper builds and validates that foundation, and shows—both theoretically and in a qualitative gradient study—that XAI on this system is feasible. Our contributions are:

- **Two differentiable VCS ports.** Independent Julia (JUTARI) and JAX (JAXTARI) implementations of the full VCS (CPU, TIA, RIOT, cartridge bank-switching, console, controllers), each with a bit-exact hard path and a differentiable soft path. Both ports are bit-for-bit identical to XITARI on all 64 supported games—64/64 byte-identical RAM and 64/64 pixel-identical screens.
- **A soft/hard formulation with theoretical analysis thereof.** We treat the ROM as a weight tensor, the RAM as a soft tape, and control flow as gates, and show that the soft forward pass is bit-exact equal to the hard forward pass at any finite temperature (Theorem 1), and give the temperature-limit error bound for the fully relaxed variant (Theorem 2).
- **An AI-assisted engineering account.** A quantified report of how an emulator port that would classically cost many person-months was built in roughly 137 hours of active work, measured from the version-control log, with large parts written autonomously by coding agents.
- **A conformance evaluation** across the 64-game Arcade Learning Environment (ALE) set, with formally defined bit- and pixel-exactness metrics, and the observation that a bit-exact re-implementation is itself an audit tool for its reference.
- **A qualitative gradient study** showing that the foundation works as intended: exact register- and ROM-byte gradients that localise to the correct hardware element when checked against the known mechanism.

2 Background and Related Work

The dominant XAI tools are *attribution* or *saliency* methods, which assign each input element a number reflecting how much it mattered to the output. Grad-CAM computes the gradient of a target output with respect to a convolutional layer and renders it as a heatmap of important regions (Selvaraju et al. 2020). Grad-CAM++ refines this with a pixel-wise weighting of positive gradients (Chattopadhyay et al. 2018). These methods are powerful and architecture-agnostic, but they share a limitation their own authors flag: there is no way to confirm that a heatmap reflects the network's true reason for deciding, because for a deep network that reason is unknown. Grad-CAM++ notes explicitly that conventional faithfulness metrics "need not correlate with the actual factors responsible for the network's decision" (Chattopadhyay et al. 2018). Validation therefore falls back on proxies—bounding-box overlap, or human-trust studies.

The problem sharpens when the object of explanation is a reinforcement learning (RL) agent rather than an image classifier. A widely studied such agent is the DQN, which learns to play Atari games from raw pixels (Mnih et al. 2013, 2015) and became a standard testbed. Greydanus et al. (2018) introduced a perturbation-based saliency for Atari agents and described strategies such as Breakout "tunneling". The one case in which they verify the saliency is an experiment where they inserted the ground-truth feature themselves. Nikulin et al. (2019) built saliency directly into the agent and validated it against human eye-tracking—a behavioural surrogate, not the machine's own computation. Atrey, Clary, and Jensen (2020) then made the consequence explicit: using counterfactual interventions (for example, mirroring the Breakout wall) they showed that popular saliency-based explanations of Atari agents are frequently unfalsifiable and do not hold up, and concluded that saliency maps are best treated as an *exploratory*, not an *explanatory*, tool. Such et al. (2019) industrialised the comparison of trained agents while leaving the validation gap untouched.

Surveys of explainable RL confirm that this is the field's structural condition rather than an incidental gap (Qing et al. 2022; Vouros 2023; Cheng, Yu, and Xing 2025; Saulières 2025). Across hundreds of methods the target is uniformly described as a closed or black box, and objective, mechanism-grounded evaluation is repeatedly named among the field's open needs. Even methods designed for interpretability inherit the problem: XDQN makes a DQN interpretable by training a transparent surrogate to mimic it, but its strongest correctness measure is fidelity *to the DQN*—agreement with another black box, not with a known mechanism (Kontogiannis and Vouros 2023). The recurring obstacle is the one Jonas and Kording (2017) identified: without a study object whose true mechanism is available, an explanation cannot be checked. We build such an object—complex, fully known, and differentiable—and validate it, opening the way to develop and validate XAI tools against a known mechanism.

Known Operators and Differentiable Programming

Our approach is related to *known-operator learning*. The idea began with deep-learning computed tomography (Würfl et al. 2016), where a known reconstruction operator is built into the network rather than learned from data. Maier et al. (2019b) later showed that embedding known, differentiable operators directly into a network reduces its maximum error bound: if a layer is known, its error term vanishes and the corresponding part of the bound cancels, even for networks of arbitrary depth. The principle is general—"any operation that allows computation of a gradient or sub-gradient towards its inputs" can be embedded (Maier et al. 2019b)—and the same spirit underlies physics-informed neural networks, which embed known governing equations (Raissi, Perdikaris, and Karniadakis 2019), and operator-aware treatments of deep learning for practitioners (Maier et al. 2019a). Our VCS is, in this sense, an extreme case: *every* operator is known, so the entire forward computation is a fixed, differentiable program, and the only thing left to "explain" is how that known program maps inputs to outputs.

Making a hand-written emulator differentiable requires a general-purpose differentiable-programming substrate. Julia provides source-to-source automatic differentiation (AD) over essentially arbitrary code via Zygote (Innes 2019), with

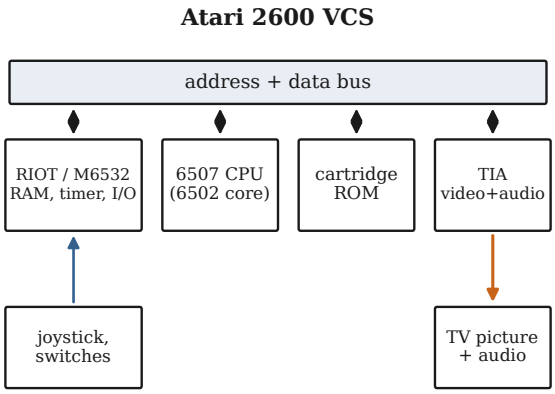

Figure 1: The Atari 2600 VCS. A 6507 CPU executes code from a (bank-switched) cartridge ROM over a shared address/data bus. The RIOT provides 128 bytes of RAM, a timer, and joystick/switch I/O, and the TIA turns register writes into a picture and sound, cycle by cycle. Every block is re-implemented and differentiated in both ports.

eager execution and mutable state that map naturally onto an emulator's registers and RAM. JAX provides trace-based AD with just-in-time compilation to XLA (Bradbury et al. 2018), which favours pure, functional, array-shaped code. The two make different trade-offs for this workload, and building both lets us cross-check each port against the other in addition to the reference. We treat the cartridge ROM as a weight tensor, the 128 bytes of RAM as a Neural-Turing-Machine-style addressable tape (Graves, Wayne, and Danihelka 2014), and discrete control flow as gates relaxed with the Gumbel-softmax reparameterisation (Jang, Gu, and Poole 2017) and straight-through estimators (Bengio, Léonard, and Courville 2013).

Why the Atari VCS Is Good Ground Truth

The VCS (Figure 1) is a real computer architecture, small enough to know completely yet complex enough to be interesting. A 6507 CPU (a cost-reduced MOS 6502) executes a program stored in the cartridge ROM. Because the address space is only 13 bits wide, larger cartridges *bank-switch*: writes to special addresses swap which 4 KB ROM page is visible. The RIOT chip (M6532) holds the 128 bytes of system RAM, an interval timer, and the input ports for the joystick, paddles, and console switches. The heart of the machine is the TIA: it has no frame buffer, so the CPU must race the electron beam, writing its registers in time with each scanline. These include the colour registers we use later—`COLUP0` and `COLUP1` for the two player sprites, `COLUPF` for the playfield, and `COLUBK` for the background—together with sprite-shape and position registers. The screen and audio are the TIA's outputs. This is the system the Arcade Learning Environment (Bellemare et al. 2013) exposes to RL agents, and that XITARI (DeepMind 2016), a fork of the Stella emulator (Stella Team 2024), uses as the reference for DQN. Because XITARI is deterministic given a ROM, an action stream, and a seed, every register, RAM byte, and pixel our ports produce can be compared against it, and any disagreement is a measurable mistake in the software port.

	JUTARI	JAXTARI
Language / AD	Julia / Zygote	JAX / XLA
Source lines	8,427	9,464
Test lines	6,052	11,788
Test cases	∼1,187	803
Opcode bytes	189	189
Cartridge mappers	9	9
RAM-exact (of 64)	**64**	**64**
Pixel-exact (of 64)	**64**	**64**
Soft throughput (steps/s)	∼363,800	∼1,666

Table 1: The two differentiable ports at a glance. Both reproduce XITARI bit-for-bit on all 64 games. JUTARI runs far faster than JAXTARI because Julia compiles to native code, whereas the JAX path dispatches each instruction through the Python interpreter and the XLA runtime (see Throughput).

3 Building Two Differentiable VCS Ports

We ported XITARI subsystem by subsystem—CPU, bus, TIA, RIOT, cartridge mappers, console, controllers, and the ALE-style environment wrapper—under a strict conformance gate. We define "correct" by three automated test harnesses: programs that drive a port and the reference on identical inputs and compare their internal state byte for byte, failing on any mismatch. The first (PXC1) compares each port against a per-instruction XITARI trace of the CPU registers, RAM, and TIA state. The second (PXC2) cross-checks the two ports against each other, so that a bug shared by the reference and one port is still caught by the second, independent implementation. The third (PXC-S) compares the full 210×160 frame buffer. Each port implements all 151 documented NMOS 6502 instructions plus the undocumented `USBC` alias and 37 common "illegal" opcodes (NOP/LAX/SAX families)—189 opcode bytes in total—in both execution modes, and nine cartridge mappers (2K, 4K, F8, F6, F4, their Superchip variants, and E0) with content-based auto-detection mirroring the reference. Table 1 summarises the two ports.

This is work that would classically be measured in person-months of expert labour. The cycle-level timing of the TIA, the sub-cycle behaviour of the RIOT timer, and the boot-time quirks of bank-switched cartridges are exactly the kind of detail that resists a clean specification and must be recovered by tracing the reference. We carried it out with coding agents operating largely autonomously under the conformance gates, committing after each step so that the version-control log doubles as a developer journal. We quantify the effort in the Results.

Hard and Soft Execution

Each port runs in two modes that share one code path (Figure 2). In **hard** mode, the emulator is the ordinary bit-exact machine: opcode bytes index an integer dispatch table, the ALU is integer arithmetic, and flags are bits. In **soft** mode,

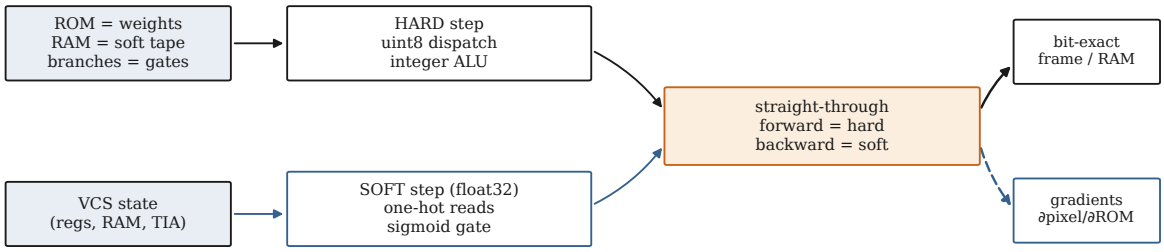

Figure 2: The two execution paths share one known mechanism. The HARD step uses integer dispatch and exact bit logic. The SOFT step reads the ROM and RAM as one-hot dot products and relaxes the branch decision with a sigmoid gate. A straight-through estimator joins them: the forward value is the hard one (bit-exact), while the backward pass uses the soft gradient, so the substrate emits both an exact frame and exact gradients of pixels with respect to ROM bytes and registers.

the same step is re-expressed so that gradients can flow. Three relaxations carry the idea.

First, every ROM and RAM read is written as a dot product against a one-hot address vector. For a memory tensor $r \in \mathbb{R}^M$ of size M (the ROM or RAM) and an address $a \in \{0, \dots, M-1\}$,

$$\mathrm{peek}(r, a) \;=\; \mathbf{1}_a^\top r \;=\; \sum_i [\![i = a]\!]\, r_i \;=\; r_a. \qquad (1)$$

The forward value is exactly r_a, but the gradient $\partial\,\mathrm{peek}/\partial r = \mathbf{1}_a$ is now defined and one-hot—this is what makes "which ROM byte explains this pixel?" a gradient question. The RAM tape uses the identical construction, which is the discrete limit of the soft, attention-style addressing of a Neural Turing Machine (Graves, Wayne, and Danihelka 2014).

Second, opcode dispatch is, in principle, a convex combination over the handler outputs. Let $\ell \in \mathbb{R}^K$ be a score vector over the $K = 256$ possible opcodes—the *logits*. In the hard machine ℓ is a one-hot indicator of the decoded opcode, and in the relaxed form it is a soft score. With temperature $T > 0$ and softmax weights $w = \mathrm{softmax}(\ell/T)$, that is $w_k = e^{\ell_k/T} / \sum_j e^{\ell_j/T}$, the dispatch output is

$$\mathrm{select}(\ell, V; T) = w^\top V, \qquad (2)$$

where the rows of V are the candidate handler outputs. This collapses to the hard pick as $T \to 0$ (Jang, Gu, and Poole 2017). In the executed step we use the saturated form directly—a hard switch on the decoded opcode—so the forward pass is exact, and the relaxed form (2) is what a fully soft variant would use. The same softmax select governs *any* discrete choice, with T setting how widely the soft pick spreads—and with it the range over which gradients flow. Figure 3 shows this on a visualisable selection (placing a target sprite among candidate screen columns rather than choosing an opcode): raising T spreads the pick over neighbouring columns, widening the *capture range* over which gradients reach the input, while the forward render stays exact.

Third, a conditional branch—"if the status flag is set, add the offset δ to the program counter (PC)"—is relaxed with a sigmoid gate. The branch condition is one processor status

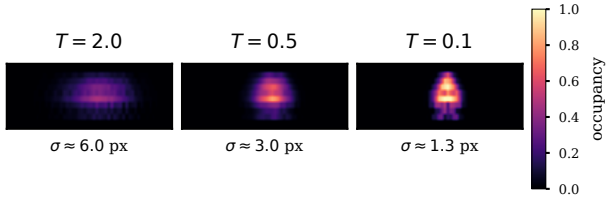

Figure 3: Soft-select temperature T in pixel space, on a visualisable selection: a target sprite's screen column chosen with $w = \mathrm{softmax}(\ell/T)$. For each T we sample the column 100 times, render the hard sprite, and average, so each pixel is the expected occupancy. Low T is a single sharp sprite (the hard pick, $\sigma \approx 1.3$ px). Raising T spreads it over neighbouring columns ($\sigma \approx 3$, then 6 px), widening the range over which gradients reach—a knob for the gradient's capture range. The forward pass stays bit-exact (Theorem 1).

flag, a single bit in the hard machine. In soft mode we let it take a real value $f \in [0, 1]$, a relaxed flag. Writing f as a logit $z = s\,(2f - 1)$, where the sign s selects branch-when-set ($s = +1$) or branch-when-clear ($s = -1$), and with sharpness α, the gate g and relaxed program counter are

$$g = \sigma(\alpha z), \qquad \mathrm{pc}_{\mathrm{soft}} = (1 - g)\,\mathrm{pc}_{\bar{b}} + g\,\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + g\,\delta, \qquad (3)$$

where $\mathrm{pc}_{\bar{b}}$ is the fall-through (not-taken) counter and $\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + \delta$ is the taken counter, so the blend simplifies to $\mathrm{pc}_{\bar{b}} + g\,\delta$. As $\alpha \to \infty$ the gate saturates to a hard branch. For the fully relaxed variant (no straight-through correction) α must be large enough that the gate's residual uncertainty $g(1 - g)$ stays below the integer-rounding threshold of pc, so the rounded counter still matches the hard branch (see the supplementary material).

These relaxations are connected to the exact machine by a *straight-through estimator* (STE) (Bengio, Léonard, and Courville 2013). For any soft quantity with a matching hard value,

$$\mathrm{STE}(\mathrm{soft}, \mathrm{hard}) = \mathrm{soft} + \mathrm{sg}(\mathrm{hard} - \mathrm{soft}), \qquad (4)$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient operator. The forward value

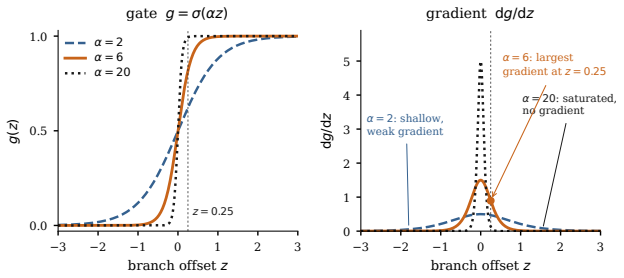

Figure 4: The soft-branch gate $g = \sigma(\alpha z)$ (left) and its gradient $\mathrm{d}g/\mathrm{d}z = \alpha\,\sigma(\alpha z)(1-\sigma(\alpha z))$ (right) for $\alpha \in \{2, 6, 20\}$, with the operating point $z{=}0.25$ marked. Too large an α (here 20) saturates the gate, leaving essentially no gradient away from the switch. Too small an α (here 2) gives a shallow gate whose gradient is weak and barely varies with z. The intermediate $\alpha{=}6$ places the operating point in the sigmoid's steep band and yields the largest, most localised gradient.

is exactly *hard*, and the backward pass uses the derivative of the soft value. The round and clamp operations are treated the same way (forward exact, backward identity inside the valid range). The consequence, made precise next, is that soft mode reproduces the hard machine *exactly* in the forward pass and differs only in the gradient it exposes.

This is the same device that makes *max-pooling* differentiable, and it applies far beyond branches. Every hard decision in the machine is a discrete *selection*: which opcode handler runs, whether a branch is taken, which object owns a pixel under TIA priority, which sprite bit lights it. Each is a switch whose winner we record in the forward pass, and the backward pass then routes the gradient to the selected value, exactly as max-pooling routes its gradient to the argmax input. With the switch stored, the entire *content* path—register values, sprite colours and graphics bits, priority compositing—is differentiable with no approximation and a bit-exact forward pass. The one quantity a stored switch cannot differentiate is a discrete *index*: the column at which a sprite is placed by strobe timing, for the same reason max-pooling gives no gradient with respect to the pooling location. There, and only there, a differentiable sampler—a sub-pixel triangular (bilinear) kernel, as in spatial transformer networks (Jaderberg et al. 2015)—restores the position gradient.

4 Soft Equals Hard: Equivalence and Gradients

We now state what the construction guarantees. Let Φ_{H} and Φ_{S} be the one-step transition maps of the hard and soft emulators, acting on the full state x (CPU registers, RAM, RIOT, TIA, and frame buffer). We state both theorems here and give only *proof sketches*. The complete, formal proofs are in the supplementary material. The first result is that the two maps agree in value.

Theorem 1 (Exact forward equivalence)**.** *For every reachable state x and every finite sharpness α and temperature T, the soft and hard one-step maps are equal by design, $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$, with identical float32 representations. Consequently, over a trajectory of N steps, $x_t^{\mathrm{S}} = x_t^{\mathrm{H}}$ for all $t \le N$. The modes differ only in the Jacobian $\partial\Phi_{\mathrm{S}}/\partial x$, which is defined and generically nonzero where $\partial\Phi_{\mathrm{H}}/\partial x$ is zero or undefined.*

Proof sketch. Each handler's forward output composes one-hot ROM/RAM reads (1), integer/round arithmetic, a hard opcode dispatch, and the straight-through branch (4), each forward-identical to the hard handler, and the stop-gradient alters only the backward rule. The one-step equality then extends to whole trajectories by induction on the step index. The complete proof, with the per-primitive equalities and the formal induction, is in the supplementary material. □

By Theorem 1, attribution in soft mode is computed on the exact hard trajectory rather than an approximation, and the forward error is identically zero independent of the temperature. The remaining question is the behaviour of a *fully* relaxed variant that does not apply the straight-through correction. For that we need a mild non-degeneracy assumption.

Assumption 1 (Decision margins)**.** *Along the executed trajectory of length N, every discrete decision has a strictly positive margin. For opcode dispatch at step t with logits $\ell^{(t)}$ and winner $k_t^\star$, the logit gap $\Delta_t = \ell^{(t)}_{k_t^\star} - \max_{k \neq k_t^\star} \ell^{(t)}_k$ is positive, and for each conditional branch with flag logit z_t, the margin $m_t = |z_t|$ is positive. Let $\Delta_{\min} = \min_t \Delta_t > 0$ and $m_{\min} = \min_t m_t > 0$.*

In the executed dispatch the flags are exact bits, so $z_t = \pm 1$ and $m_{\min} = 1$, so Assumption 1 holds trivially for branches and is a statement only about ties in a fully soft dispatch.

Theorem 2 (Temperature-limit bound)**.** *Consider the fully relaxed emulator that uses the softmax dispatch* (2) *and sigmoid branch* (3) without *the straight-through correction. Under Assumption 1, its one-step map converges to the hard map as $T \to 0$ and $\alpha \to \infty$, with per-step deviation*

$$\left\|\Phi_{\mathrm{S}}^{T}(x) - \Phi_{\mathrm{H}}(x)\right\|_{\infty} \;\le\; C\Big[(K-1)\,e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}\Big], \tag{5}$$

where C bounds the value range (bytes ≤ 255, branch offsets ≤ 256). Over N steps, with Lipschitz constant $L \ge 1$ for error propagation, the trajectory deviation is at most $\frac{L^N-1}{L-1}\,C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}] = \mathcal{O}(e^{-c/T})$ with $c = \Delta_{\min}$.

Proof sketch. Bound the non-winning softmax mass by $(K-1)e^{-\Delta_{\min}/T}$ and the sigmoid error by $e^{-\alpha m_{\min}}$, then propagate over N steps with the Lipschitz constant L. The complete proof is in the supplementary material. □

Corollary 1. *The straight-through estimator* (4) *attains the $T \to 0$ forward limit of Theorem 2* exactly at finite temperature*: it sets the bracket in* (5) *to zero for all T, α while keeping the same backward Jacobian as the limit. Theorem 1 is thus the $T \to 0$ forward limit, achieved by construction.*

These results are matched by the implementation's unit tests, which confirm that the softmax dispatch saturates to the exact hard pick at large logits, that the sigmoid branch

saturates to the exact taken/not-taken program counter at large α, and that the soft memory read collapses to ordinary indexing at small T—the empirical face of Theorems 1–2.

5 Evaluation Methodology

We report five families of measurements, defined here so the results are unambiguous. The two conformance metrics are evaluated on the 64 ALE games playable by the original DQN benchmark: for each game we run the standard ALE boot (60 no-op frames plus four resets) and then a fixed action stream. A game is *RAM-exact* if its 128-byte RIOT RAM is identical to XITARI on every frame, and *pixel-exact* if its full 210×160 frame buffer (palette indices) is identical on every frame. We report the count of each out of 64. We estimate *active* implementation wall-time from the version-control log, treating any gap between consecutive commits longer than three hours as a session boundary and summing the active sessions, with the two- and four-hour thresholds reported as a sensitivity check. We count authored test cases per port and the distinct numbered bug investigations recorded in the development log. Finally, we measure soft-mode throughput as steps per second on a fixed ROM after just-in-time warm-up, on a single Apple M1 Max core (JAX CPU backend, Julia 1.10), reporting the median of repeated runs.

6 Results

Both ports are bit-for-bit identical to XITARI on *all* 64 ALE games: 64/64 games are RAM-exact (zero differing bytes across every frame) and 64/64 are pixel-exact (zero differing pixels across every frame). Reaching full pixel exactness required porting XITARI's actual per-colour-clock TIA object model rather than approximating its output.

Effort: Person-Months into Days

The project comprises 373 commits over a 29-day calendar span (a cumulative commit-time plot is given in the supplementary material). To estimate active implementation time we split the commit stream into work sessions. We use a three-hour gap between consecutive commits as the session boundary: within a session commits are minutes apart (the median is one every $\sim$13 minutes), whereas idle periods last many hours, so three hours sits well above normal working pauses and below the idle stretches. The split is therefore insensitive to the exact value—106 active hours at a two-hour cutoff, 162 at four hours. With the three-hour boundary, the 373 commits form 52 active sessions totalling **136.8 hours**, or about **5.7 working days**. The remaining $\sim 80\%$ of the calendar span was idle and excluded: it consists mostly of stretches when the lead programmer was travelling and without reliable internet access. The work used Anthropic's Claude Opus 4.7 and Opus 4.8 as the primary coding models, with a brief autonomous stint by Fable 5. Against this we built two independent emulators totalling $\sim$17,900 lines of source and a comparable volume of tests ($\sim$1,990 test cases across the two ports), and chased 67 distinct numbered bug investigations to a bit-exact result.

For a baseline, XITARI is a C++ core of about 46,000 lines derived from Stella, a project with sources dating to the 1990s and an xitari fork spanning 2014–2017 (DeepMind 2016; Stella Team 2024)—many person-years of cumulative work. Our two ports reproduce its behaviour (bit-exact RAM, pixel-exact screens) in about 17,900 lines written in 137 active hours. We do not claim a controlled comparison—the reference is larger in scope (audio, every cartridge type, tooling)—but the contrast between a multi-year reference codebase and a six-working-day reimplementation is the right order of magnitude for what agentic tools change.

Throughput

On the soft-mode benchmark, JUTARI sustains $\sim$363,800 steps per second and JAXTARI $\sim$1,666, a $\sim 218\times$ gap (Table 1). The difference is dominated by per-operation kernel-launch overhead in JAX: each soft step dispatches one opcode through a 256-way switch, and at single-instruction granularity the per-launch overhead far exceeds the instruction itself, whereas Julia inlines the small handlers. This is a property of the execution granularity, not the AD systems: for a gradient-based XAI workflow (one forward-plus-backward pass per attribution) JAXTARI's $\sim$30 s per 50,000-step trace is comfortably interactive, and for batched, training-scale rollouts the JAX implementation is far more efficient once many environments are vectorised in parallel.

Proof of Concept: Ground-Truth Gradients

The point of the foundation is that gradients computed on it can be *checked*. Because soft mode is forward-exact (Theorem 1), it yields exact gradients of any read-out with respect to the state and the ROM. We give a small qualitative study (Figure 5). It is a proof of concept that the substrate behaves as intended, not an XAI evaluation.

We differentiate the rendered frame with respect to the TIA colour registers: $\partial\text{screen}/\partial\texttt{COLUP0}$ localises exactly to the cannon (player 0), $\partial\text{screen}/\partial\texttt{COLUP1}$ to the row of invaders (player 1), and $\partial\text{screen}/\partial\texttt{COLUBK}$ to the background—each pixel attributing to the exactly correct register, a statement that can be *checked* here and cannot be on a black box. The stored-switch view extends this: a single cannon *graphics* bit, once represented as a value behind its switch, has a non-zero gradient to precisely the pixels it lights.

A naive gradient from the joystick to the screen is zero, because the path runs through a discrete sprite *position* index. This is not a failure of differentiation but the ground truth telling us that a naive gradient saliency would wrongly report "the joystick does not affect the screen." Applying the differentiable sampler to the position makes the relationship visible: the gradient of the on-screen cannon position with respect to the four joystick directions correctly recovers the control mapping (push RIGHT, and not up/down, to slide the cannon toward the target invader), and the per-direction $\partial\text{screen}/\partial\text{joystick}$ map highlights the cannon edges that move. We can *verify* this against the known wiring—which is the whole point.

7 Discussion

This paper delivers four results. We built two independent differentiable ports of the Atari VCS, in Julia and JAX, that

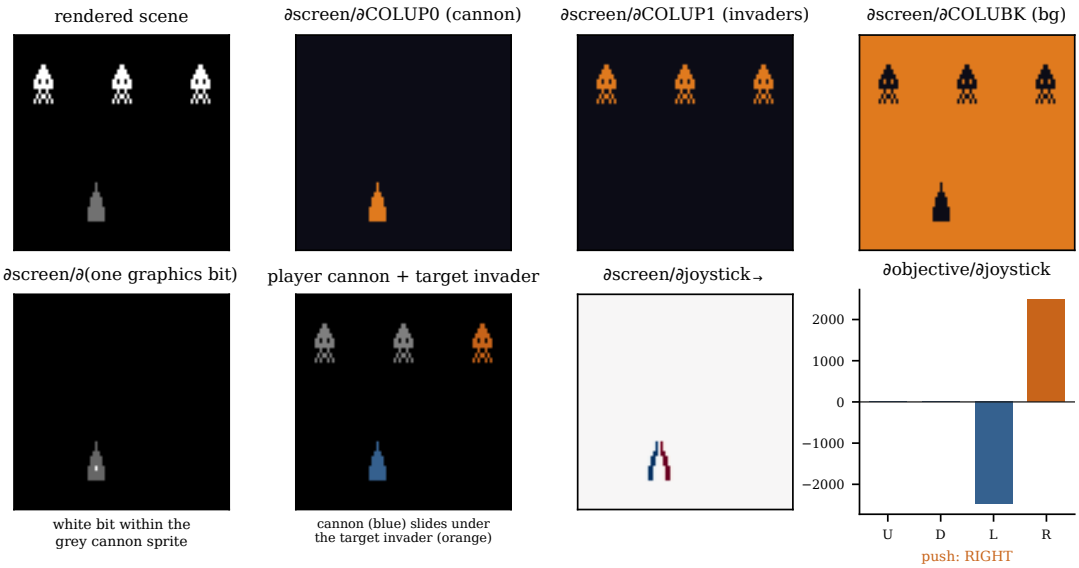

Figure 5: Qualitative XAI on a Space-Invaders-style scene in the differentiable VCS: A row of invaders (player 1) and a player cannon (player 0)—rather than a trace of the game ROM. **Top:** differentiating jutari's soft TIA renderer by reverse-mode AD gives a Grad-CAM analogue, $\partial\text{screen}/\partial\text{register}$, which localises each object to exactly the pixels its register controls (the cannon via `COLUP0`, the invaders via `COLUP1`, the background via `COLUBK`)—a localisation that can be *checked* against ground truth. **Bottom, left to right:** a single cannon graphics bit (bright, within the faint cannon outline) attributes to its own pixels once represented as a value behind a stored switch. Next, the scene with the cannon and the target invader it should slide under (orange). Then the screen's directional derivative with respect to each joystick input, where a differentiable sampler over the sprite position—not the controller-to-CPU path—makes the position differentiable, highlighting the cannon edges that move. Finally, the gradient of the on-screen cannon position with respect to the four joystick directions, which recovers the control mapping: pushing RIGHT, and not up/down, slides the cannon toward the target.

are bit-exact in RAM and pixel-exact on screen against XITARI on all 64 DQN games. We proved that the soft forward pass equals the hard one exactly, so gradients are computed on the true execution rather than an approximation. We measured the implementation effort at about 137 active hours, built largely by coding agents—a concrete data point on how quickly such a system can now be produced. And we showed qualitatively that gradients localise to the correct hardware and can be checked against the known mechanism. Theoretic analysis of α and T showed that exact operation is also possible with the soft version in the forward pass for small T and large α.

Building the ports also revealed a bug in the reference. Driving the ports to 64/64 pixel exactness surfaced a latent non-determinism in XITARI itself. One title's attract-mode demo reads *uninitialised* on-cartridge Superchip RAM as a cheap random-number source. XITARI seeds that RAM from `time(NULL)` and ignores its own seed parameter, so the title renders differently on every run despite a pinned seed. This divergence is invisible to RAM-state checks and shows up only on screen. We reach full conformance by pinning the seed deterministically in both emulators (also making the suite reproducible), and have prepared an upstream fix—a bit-exact re-implementation is useful in both directions: the reference validates the port, and the port audits the reference for reproducibility bugs.

The hardest divergences were all in timing—the RIOT timer's reset state, the TIA's sub-cycle object rendering, and the boot-time format probe that seeds free-running counters games never re-initialise. Two model anecdotes: the bulk of the work was carried out by Claude Opus 4.7 and 4.8, with one autonomous session driven by Fable 5. An early attempt with Kimi-K2.6 (a one-trillion-parameter model) ended when it refused to start, judging the port to be months of work.

One limitation: the effort comparison is an order-of-magnitude estimate, not a controlled trial. This does not affect the core claims—the forward equivalence of Theorem 1 and the measured 64/64 conformance of both ports.

8 Conclusion

We set out to give explainable AI a study object that escapes the known-but-trivial / complex-but-unknown dichotomy: a genuine, complex information-processing system that is at once fully known and fully differentiable. We built it twice, in Julia and JAX, validated both ports bit-for-bit against the reference on all 64 supported Atari games, and proved the differentiable forward pass bit-exact equal to the original machine while exposing gradients where bit logic has none. With the mechanism fully known, an attribution can now—for the first time on a system of this complexity—be scored right or wrong against the truth. We hope the community

takes up this system as a testbed, both to evaluate existing XAI methods against a known mechanism and to develop new ones.

Ethical Statement

This work involves no human subjects, personal data, or sensitive information, and uses Atari ROMs only as fixed computational inputs. The agentic method we report is dual-use, but pairing autonomy with a strict, automatically checked conformance oracle keeps it safe.

References

Atrey, A.; Clary, K.; and Jensen, D. 2020. Exploratory Not Explanatory: Counterfactual Analysis of Saliency Maps for Deep Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The Arcade Learning Environment: An Evaluation Platform for General Agents. *Journal of Artificial Intelligence Research*, 47: 253–279.

Bengio, Y.; Léonard, N.; and Courville, A. 2013. Estimating or Propagating Gradients through Stochastic Neurons for Conditional Computation. arXiv:1308.3432.

Bezanson, J.; Edelman, A.; Karpinski, S.; and Shah, V. B. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59(1): 65–98.

Bradbury, J.; Frostig, R.; Hawkins, P.; Johnson, M. J.; Leary, C.; Maclaurin, D.; Necula, G.; Paszke, A.; VanderPlas, J.; Wanderman-Milne, S.; and Zhang, Q. 2018. JAX: Composable Transformations of Python+NumPy Programs. http://github.com/google/jax. Software.

Chattopadhyay, A.; Sarkar, A.; Howlader, P.; and Balasubramanian, V. N. 2018. Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 839–847. IEEE.

Cheng, Z.; Yu, J.; and Xing, X. 2025. A Survey on Explainable Deep Reinforcement Learning. *arXiv preprint arXiv:2502.06869*.

DeepMind. 2016. Xitari: An Arcade Learning Environment Fork. https://github.com/google-deepmind/xitari. Accessed: 2026-06-16.

Graves, A.; Wayne, G.; and Danihelka, I. 2014. Neural Turing Machines. arXiv:1410.5401.

Greydanus, S.; Koul, A.; Dodge, J.; and Fern, A. 2018. Visualizing and Understanding Atari Agents. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, 1792–1801. PMLR.

Innes, M. 2019. Don't Unroll Adjoint: Differentiating SSA-Form Programs. arXiv:1810.07951.

Jaderberg, M.; Simonyan, K.; Zisserman, A.; and Kavukcuoglu, K. 2015. Spatial Transformer Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28.

Jang, E.; Gu, S.; and Poole, B. 2017. Categorical Reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*.

Jonas, E.; and Kording, K. P. 2017. Could a Neuroscientist Understand a Microprocessor? *PLOS Computational Biology*, 13(1): e1005268.

Kontogiannis, A.; and Vouros, G. A. 2023. XDQN: Inherently Interpretable DQN through Mimicking. arXiv:2301.03043.

Maier, A.; Syben, C.; Lasser, T.; and Riess, C. 2019a. A Gentle Introduction to Deep Learning in Medical Image Processing. *Zeitschrift für Medizinische Physik*, 29(2): 86–101.

Maier, A. K.; Syben, C.; Stimpel, B.; Würfl, T.; Hoffmann, M.; Schebesch, F.; Fu, W.; Mill, L.; Kling, L.; and Christiansen, S. 2019b. Learning with Known Operators Reduces Maximum Error Bounds. *Nature Machine Intelligence*, 1(8): 373–380.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. 2013. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Nikulin, D.; Ianina, A.; Aliev, V.; and Nikolenko, S. 2019. Free-Lunch Saliency via Attention in Atari Agents. arXiv:1908.02511.

Qing, Y.; Liu, S.; Song, J.; Zhou, Y.; Chen, K.; Wang, H.; and Song, M. 2022. A Survey on Explainable Reinforcement Learning: Concepts, Algorithms, and Challenges. *arXiv preprint arXiv:2211.06665*.

Raissi, M.; Perdikaris, P.; and Karniadakis, G. E. 2019. Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *Journal of Computational Physics*, 378: 686–707.

Saulières, L. 2025. A Survey of Explainable Reinforcement Learning: Targets, Methods and Needs. *arXiv preprint arXiv:2507.12599*.

Selvaraju, R. R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; and Batra, D. 2020. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *International Journal of Computer Vision*, 128(2): 336–359. Originally arXiv:1610.02391 (2016).

Stella Team. 2024. Stella: A Multi-Platform Atari 2600 VCS Emulator. https://stella-emu.github.io. Accessed: 2026-06-16.

Such, F. P.; Madhavan, V.; Liu, R.; Wang, R.; Castro, P. S.; Li, Y.; Zhi, J.; Schubert, L.; Bellemare, M. G.; Clune, J.; and Lehman, J. 2019. An Atari Model Zoo for Analyzing, Visualizing, and Comparing Deep Reinforcement Learning Agents. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 3260–3267.

Vouros, G. A. 2023. Explainable Deep Reinforcement Learning: State of the Art and Challenges. *ACM Computing Surveys*, 55(5): 1–39.

Würfl, T.; Ghesu, F. C.; Christlein, V.; and Maier, A. 2016. Deep Learning Computed Tomography. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 432–440. Springer.